

```
"""
Secure Chat - FastAPI backend
- Email + password signup/login (JWT auth)
- Messages encrypted at rest with AES-256-
GCM
- SQLite storage via SQLAlchemy
"""

import os
import base64
import datetime
from typing import List

from fastapi import FastAPI, Depends,
HTTPException, status
from fastapi.security import
OAuth2PasswordBearer,
OAuth2PasswordRequestForm
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse
from pydantic import BaseModel, EmailStr

from sqlalchemy import create_engine,
Column, Integer, String, DateTime,
ForeignKey
from sqlalchemy.orm import
declarative_base, sessionmaker, Session,
relationship

from passlib.context import CryptContext
```

```

from jose import jwt, JWTError

from
cryptography.hazmat.primitives.ciphers.aead
import AESGCM

# -----
# -----
# Config
# -----
# -----

SECRET_KEY = os.environ.get("JWT_SECRET",
"dev-secret-change-me")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 # 1
day

# 32-byte key for AES-256-GCM. In
production, set MESSAGE_ENC_KEY as a
# base64-encoded 32-byte value in Replit
"Secrets". If missing, a key is
# generated at startup (messages will not
survive a restart in that case).
_raw_key =
os.environ.get("MESSAGE_ENC_KEY")
if _raw_key:
    ENC_KEY = base64.b64decode(_raw_key)
else:
    ENC_KEY =
AESGCM.generate_key(bit_length=256)

DATABASE_URL = "sqlite:///./secure_chat.db"

```

```

# -----
# Database setup
# -----

engine = create_engine(DATABASE_URL,
connect_args={"check_same_thread": False})
SessionLocal =
sessionmaker(autocommit=False,
autoflush=False, bind=engine)
Base = declarative_base()

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True,
index=True)
    email = Column(String, unique=True,
index=True, nullable=False)
    username = Column(String, unique=True,
index=True, nullable=False)
    phone = Column(String, unique=True,
index=True, nullable=True)
    hashed_password = Column(String,
nullable=False)
    created_at = Column(DateTime,
default=datetime.datetime.utcnow)

    sent_messages = relationship(
        "Message",
foreign_keys="Message.sender_id",
back_populates="sender"

```

```

    )

class Message(Base):
    __tablename__ = "messages"

    id = Column(Integer, primary_key=True,
index=True)
    sender_id = Column(Integer,
ForeignKey("users.id"), nullable=False)
    receiver_email = Column(String,
index=True, nullable=False)
    nonce = Column(String,
nullable=False)      # base64
    ciphertext = Column(String,
nullable=False) # base64
    created_at = Column(DateTime,
default=datetime.datetime.utcnow)

    sender = relationship("User",
foreign_keys=[sender_id],
back_populates="sent_messages")

Base.metadata.create_all(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

```

# -----
# -----
# Security helpers
# -----
# -----

pwd_context = CryptContext(schemes=
["bcrypt"], deprecated="auto")
oauth2_scheme =
OAuth2PasswordBearer(tokenUrl="/api/login")

def hash_password(password: str) -> str:
    # bcrypt has a hard 72-byte limit;
    truncate safely on the raw bytes
    truncated = password.encode("utf-8")
    [:72].decode("utf-8", errors="ignore")
    return pwd_context.hash(truncated)

def verify_password(plain: str, hashed:
str) -> bool:
    truncated = plain.encode("utf-8")
    [:72].decode("utf-8", errors="ignore")
    return pwd_context.verify(truncated,
hashed)

def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.datetime.utcnow() +
datetime.timedelta(
    minutes=ACCESS_TOKEN_EXPIRE_MINUTES

```

```

    )
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode,
SECRET_KEY, algorithm=ALGORITHM)

def encrypt_message(plaintext: str) ->
tuple[str, str]:
    aesgcm = AESGCM(ENC_KEY)
    nonce = os.urandom(12)
    ct = aesgcm.encrypt(nonce,
plaintext.encode("utf-8"), None)
    return
base64.b64encode(nonce).decode(),
base64.b64encode(ct).decode()

def decrypt_message(nonce_b64: str, ct_b64:
str) -> str:
    aesgcm = AESGCM(ENC_KEY)
    nonce = base64.b64decode(nonce_b64)
    ct = base64.b64decode(ct_b64)
    return aesgcm.decrypt(nonce, ct,
None).decode("utf-8")

def get_current_user(token: str =
Depends(oauth2_scheme), db: Session =
Depends(get_db)) -> User:
    credentials_exception = HTTPException(

status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate
credentials",

```

```

        headers={"WWW-Authenticate":
"Bearer"},
    )
    try:
        payload = jwt.decode(token,
SECRET_KEY, algorithms=[ALGORITHM])
        email: str = payload.get("sub")
        if email is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = db.query(User).filter(User.email
== email).first()
    if user is None:
        raise credentials_exception
    return user

# -----
# -----
# Pydantic schemas
# -----
# -----

class SignupRequest(BaseModel):
    email: EmailStr
    username: str
    phone: str | None = None
    password: str

class MessageOut(BaseModel):
    id: int

```

```

    sender_email: str
    sender_username: str
    receiver_email: str
    text: str
    created_at: datetime.datetime

class SendMessageRequest(BaseModel):
    receiver_email: EmailStr
    text: str

# -----
# -----
# App
# -----
# -----

app = FastAPI(title="Secure Chat")

@app.post("/api/signup", status_code=201)
def signup(payload: SignupRequest, db:
Session = Depends(get_db)):
    existing =
db.query(User).filter(User.email ==
payload.email).first()
    if existing:
        raise
HTTPException(status_code=400,
detail="Email already registered")

    username = payload.username.strip()
    if len(username) < 3:

```



```

        raise HTTPException(
            status_code=400,
            detail="Username must be at least 3
characters"
        )

        existing_username =
db.query(User).filter(User.username ==
username).first()
        if existing_username:
            raise
HTTPException(status_code=400,
detail="Username already taken")

        phone = payload.phone.strip() if
payload.phone else None
        if phone:
            existing_phone =
db.query(User).filter(User.phone ==
phone).first()
            if existing_phone:
                raise
HTTPException(status_code=400,
detail="Phone number already registered")

        if len(payload.password) < 6:
            raise HTTPException(
                status_code=400,
                detail="Password must be at least 6
characters"
            )

        user = User(
            email=payload.email,

```

```

        username=username,
        phone=phone,

hashed_password=hash_password(payload.password),
    )
    db.add(user)
    db.commit()
    db.refresh(user)

    token = create_access_token({"sub":
user.email})
    return {
        "access_token": token,
        "token_type": "bearer",
        "email": user.email,
        "username": user.username,
        "phone": user.phone,
    }

@app.post("/api/login")
def login(form_data:
OAuth2PasswordRequestForm = Depends(), db:
Session = Depends(get_db)):
    # form_data.username holds whatever the
user typed in the login box:
    # it can be either their email or their
phone number.
    identifier = form_data.username.strip()
    user = (
        db.query(User)
        .filter((User.email == identifier)
| (User.phone == identifier))

```

```

        .first()
    )
    if not user or not
verify_password(form_data.password,
user.hashed_password):
        raise HTTPException(

status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect email or
password",
            headers={"WWW-Authenticate":
"Bearer"},
        )

    token = create_access_token({"sub":
user.email})
    return {
        "access_token": token,
        "token_type": "bearer",
        "email": user.email,
        "username": user.username,
        "phone": user.phone,
    }

@app.get("/api/me")
def read_me(current_user: User =
Depends(get_current_user)):
    return {
        "email": current_user.email,
        "username": current_user.username,
        "phone": current_user.phone,
    }

```

```

@app.post("/api/messages", status_code=201)
def send_message(
    payload: SendMessageRequest,
    current_user: User =
Depends(get_current_user),
    db: Session = Depends(get_db),
):
    recipient =
db.query(User).filter(User.email ==
payload.receiver_email).first()
    if not recipient:
        raise
    HTTPException(status_code=404,
detail="Recipient not found")

    nonce_b64, ct_b64 =
encrypt_message(payload.text)
    msg = Message(
        sender_id=current_user.id,

receiver_email=payload.receiver_email,
        nonce=nonce_b64,
        ciphertext=ct_b64,
    )
    db.add(msg)
    db.commit()
    db.refresh(msg)
    return {"id": msg.id, "created_at":
msg.created_at}

@app.get("/api/messages/{other_email}",
response_model=List[MessageOut])

```

```

def get_conversation(
    other_email: str,
    current_user: User =
Depends(get_current_user),
    db: Session = Depends(get_db),
):
    other =
db.query(User).filter(User.email ==
other_email).first()
    if not other:
        raise
HTTPException(status_code=404, detail="User
not found")

    rows = (
        db.query(Message)
        .filter(
            (
                (Message.sender_id ==
current_user.id)
                & (Message.receiver_email
== other.email)
            )
            | (
                (Message.sender_id ==
other.id)
                & (Message.receiver_email
== current_user.email)
            )
        )
        .order_by(Message.created_at.asc())
        .all()
    )

```

```

        result = []
        for row in rows:
            try:
                text =
decrypt_message(row.nonce, row.ciphertext)
            except Exception:
                text = "[unable to decrypt]"
            result.append(
                MessageOut(
                    id=row.id,

sender_email=row.sender.email,

sender_username=row.sender.username,

receiver_email=row.receiver_email,
                    text=text,
                    created_at=row.created_at,
                )
            )
        return result

# -----
# -----
# Static frontend
# -----
# -----

app.mount("/static",
StaticFiles(directory="static"),
name="static")

```

```
@app.get("/icon-192.png")
def icon_192():
    return FileResponse("icon-192.png")

@app.get("/icon-512.png")
def icon_512():
    return FileResponse("icon-512.png")

@app.get("/")
def index():
    return
    FileResponse("static/index.html")

if __name__ == "__main__":
    import uvicorn

    port = int(os.environ.get("PORT",
8000))
    uvicorn.run("main:app", host="0.0.0.0",
port=port, reload=False)
```